Frontlines Edutech

https://www.frontlinesedutech.com

SQL Learning Guide

WHERE Clause | DCL | TCL | User Roles & Permissions

Topics Covered

#	Topic
1	WHERE Clause — AND, OR, NOT, IN, BETWEEN, LIKE
2	DCL — Data Control Language (GRANT, REVOKE)
3	User Roles and Permissions
4	TCL — Transaction Control Language (COMMIT, ROLLBACK)
5	Real-Time Interview Questions & Answers
6	Quick Revision Cheat Sheet

1. WHERE Clause — Filtering Data

Definition

The WHERE clause is used with SELECT, UPDATE, and DELETE statements to filter rows based on a condition.
Only rows that match the condition are returned or affected.
Without WHERE, ALL rows in the table are affected!

1.1 AND Operator

Definition

AND returns rows only when ALL conditions are true. Both conditions must be satisfied.

Syntax

```
SELECT * FROM table_name
WHERE condition1 AND condition2;
```

Example

```
-- Get students from Hyderabad who have a Gmail address
SELECT * FROM flmstudents
WHERE address = 'Hyderabad' AND emailid LIKE '%gmail.com';
```

Output

```
+-----------+--------------+--------------------+-----------+
| studentid | studentname  | emailid            | address   |
+-----------+--------------+--------------------+-----------+
| 1         | Rahul Sharma | rahul@gmail.com    | Hyderabad |
+-----------+--------------+--------------------+-----------+
```

Quick Tip

Use AND when you need MULTIPLE conditions to be true at the same time.

1.2 OR Operator

Definition

OR returns rows when AT LEAST ONE of the conditions is true. If either condition matches, the row is included.

Syntax

```
SELECT * FROM table_name
WHERE condition1 OR condition2;
```

Example

```
-- Get students from Hyderabad OR Mumbai
SELECT * FROM flmstudents
WHERE address = 'Hyderabad' OR address = 'Mumbai';
```

Output

```
+-----------+-------------+--------------------+-----------+
| studentid | studentname  | emailid            | address   |
+-----------+-------------+--------------------+-----------+
| 1         | Rahul Sharma | rahul@gmail.com    | Hyderabad |
| 2         | Priya Singh  | priya@gmail.com    | Mumbai    |
+-----------+-------------+--------------------+-----------+
```

1.3 NOT Operator

Definition

NOT reverses (negates) the result of a condition. It returns rows where the condition is FALSE.

Syntax

```
SELECT * FROM table_name
WHERE NOT condition;
```

Example

```
-- Get all students who are NOT from Delhi
SELECT * FROM flmstudents
WHERE NOT address = 'Delhi';
```

Output

```
+-----------+-------------+--------------------+-----------+
| studentid | studentname  | emailid            | address   |
+-----------+-------------+--------------------+-----------+
| 1         | Rahul Sharma | rahul@gmail.com    | Hyderabad |
| 2         | Priya Singh  | priya@gmail.com    | Mumbai    |
+-----------+-------------+--------------------+-----------+
(Amit Kumar from Delhi is excluded)
```

1.4 IN Operator

Definition

IN checks if a value matches any value in a given list.
It is a shortcut for writing multiple OR conditions.

Syntax

```
SELECT * FROM table_name
WHERE column_name IN (value1, value2, value3);
```

Example

```
-- Get students from Hyderabad, Mumbai, or Delhi
SELECT * FROM flmstudents
WHERE address IN ('Hyderabad', 'Mumbai', 'Delhi');

-- NOT IN example: exclude these cities
SELECT * FROM flmstudents
WHERE address NOT IN ('Chennai', 'Pune');
```

Output (IN example)

```
All 3 students are returned (all match one of the listed cities).
```

Quick Tip

IN is cleaner than writing: address = 'Hyderabad' OR address = 'Mumbai' OR address = 'Delhi'

Use NOT IN to exclude a list of values.

1.5 BETWEEN Operator

Definition

BETWEEN filters rows within a range of values (inclusive — includes both the start and end values).

It works with numbers, dates, and text.

Syntax

```
SELECT * FROM table_name
WHERE column_name BETWEEN value1 AND value2;
```

Example — Numbers

```
-- Get students with marks between 60 and 90
SELECT * FROM students
WHERE marks BETWEEN 60 AND 90;
```

Example — Dates

```
-- Get records created in January 2024
SELECT * FROM orders
WHERE order_date BETWEEN '2024-01-01' AND '2024-01-31';
```

IMPORTANT

BETWEEN is INCLUSIVE: BETWEEN 60 AND 90 includes rows where marks = 60 AND marks = 90.
Use NOT BETWEEN to get values outside the range.

1.6 LIKE Operator (Pattern Matching)

Definition

LIKE is used to search for a specific pattern in a column.
It uses wildcards:
% (percent) — represents zero or more characters
_ (underscore) — represents exactly ONE character

Syntax

```
SELECT * FROM table_name
WHERE column_name LIKE 'pattern';
```

Examples

```
-- Names starting with 'R'
SELECT * FROM flmstudents WHERE studentname LIKE 'R%';

-- Names ending with 'a'
SELECT * FROM flmstudents WHERE studentname LIKE '%a';

-- Names containing 'ul' anywhere
SELECT * FROM flmstudents WHERE studentname LIKE '%ul%';

-- Exactly 5 characters long
SELECT * FROM flmstudents WHERE studentname LIKE '_____';

-- Names starting with 'Pr' and at least 3 characters
SELECT * FROM flmstudents WHERE studentname LIKE 'Pr_%';
```

Pattern	Matches
LIKE 'R%'	Starts with R (Rahul, Ravi, Ram...)
LIKE '%a'	Ends with a (Priya, Meera...)
LIKE '%ul%'	Contains 'ul' anywhere (Rahul, Mukul...)
LIKE '_____'	Exactly 5 characters
LIKE 'R_h_l'	R then any char then h then any char then l

2. DCL — Data Control Language

Definition

DCL (Data Control Language) controls who can access data in a database.
It manages user permissions and security.
DCL has two main commands: GRANT and REVOKE.

2.1 GRANT — Give Permissions

Definition

GRANT gives a user (or role) permission to perform specific operations on a database object.
Without a GRANT, users cannot access tables or run queries.

Syntax

```
GRANT permission_type ON object_name TO user_name;

-- Common permission types:
-- SELECT, INSERT, UPDATE, DELETE, EXECUTE, ALL PRIVILEGES
```

Examples

```
-- Give user 'john' permission to read (SELECT) the students table
GRANT SELECT ON flmstudents TO john;

-- Give user 'manager' ability to read and modify data
GRANT SELECT, INSERT, UPDATE ON flmstudents TO manager;

-- Give ALL permissions on the table
GRANT ALL PRIVILEGES ON flmstudents TO admin_user;

-- Grant permission on the entire database (SQL Server)
GRANT SELECT ON DATABASE::FLM TO analyst_user;
```

Output

```
Grant succeeded.
User 'john' can now run: SELECT * FROM flmstudents;
```

2.2 REVOKE — Remove Permissions

Definition

REVOKE takes away previously granted permissions from a user.

After REVOKE, the user can no longer perform that operation.

Syntax

```
REVOKE permission_type ON object_name FROM user_name;
```

Examples

```
-- Remove SELECT permission from john
REVOKE SELECT ON flmstudents FROM john;

-- Remove INSERT and UPDATE from manager
REVOKE INSERT, UPDATE ON flmstudents FROM manager;

-- Remove ALL permissions from a user
REVOKE ALL PRIVILEGES ON flmstudents FROM old_user;
```

Output

```
Revoke succeeded.
User 'john' can no longer run: SELECT * FROM flmstudents;
```

WARNING

Always be careful with GRANT ALL PRIVILEGES — only give admin-level access to trusted users.
Use the principle of least privilege: give users only the minimum permissions they need.

2.3 User Roles and Permissions

Definition

A Role is a named collection of permissions that can be assigned to multiple users.
Instead of granting permissions to each user individually, you create a role, give it permissions,
then assign the role to users. This makes management much easier.

Creating and Using Roles (SQL Server)

```
-- Step 1: Create a role
CREATE ROLE student_viewer;

-- Step 2: Grant permissions to the role
GRANT SELECT ON flmstudents TO student_viewer;

-- Step 3: Add a user to the role
ALTER ROLE student_viewer ADD MEMBER john;
ALTER ROLE student_viewer ADD MEMBER priya;
```

```
-- Step 4: Remove a user from a role
ALTER ROLE student_viewer DROP MEMBER john;
```

Common Built-in Roles (SQL Server)

Role Name	Description
db_owner	Full control over the database
db_datareader	Can read (SELECT) all tables
db_datawriter	Can add, modify, and delete data
db_ddladmin	Can create, modify, drop tables (DDL)
public	Default minimal access for all users

Command	Purpose	Example
GRANT	Give permissions to user/role	GRANT SELECT ON tbl TO user1
REVOKE	Remove permissions from user/role	REVOKE SELECT ON tbl FROM user1
DENY	Explicitly block access (overrides GRANT)	DENY DELETE ON tbl TO user1

3. TCL — Transaction Control Language

Definition

TCL (Transaction Control Language) controls database transactions.
A Transaction is a group of SQL statements that are executed together as a single unit.
Either ALL statements succeed (commit) or NONE of them take effect (rollback).
TCL ensures data consistency and integrity in the database.

Think of a transaction like a bank transfer: deducting money from Account A AND adding it to Account B must both succeed — or neither should happen!

3.1 COMMIT — Save Changes Permanently

Definition

COMMIT permanently saves all changes made in the current transaction to the database.
After a COMMIT, the changes cannot be undone using ROLLBACK.
In SQL Server, each statement auto-commits by default unless you use BEGIN TRANSACTION.

Syntax

```
BEGIN TRANSACTION;
  -- your SQL statements here
COMMIT;

-- Or simply:
COMMIT TRANSACTION;
```

Example

```
BEGIN TRANSACTION;

  -- Step 1: Insert a new student
  INSERT INTO flmstudents (studentid, studentname, emailid, address)
  VALUES (4, 'Neha Gupta', 'neha@gmail.com', 'Pune');

  -- Step 2: Update another student
  UPDATE flmstudents
  SET address = 'Bangalore'
  WHERE studentid = 2;

COMMIT;   -- Both changes are now permanently saved
```

Output

```
Transaction committed successfully.
New student added + address updated — both changes are now permanent.
```

3.2 ROLLBACK — Undo Changes

Definition

ROLLBACK undoes all changes made in the current transaction, reverting the database to its state before the transaction began.
ROLLBACK only works BEFORE a COMMIT — once committed, changes cannot be rolled back.

Syntax

```
BEGIN TRANSACTION;
  -- your SQL statements here
ROLLBACK;   -- undo everything in this transaction
```

Example — Catching a Mistake

```
BEGIN TRANSACTION;

  -- Oops! Accidentally trying to delete ALL students (forgot WHERE!)
  DELETE FROM flmstudents;

  -- Wait... that's wrong! Let's undo it immediately
ROLLBACK;   -- All rows are restored, nothing is deleted

-- Correct approach: delete only one student
BEGIN TRANSACTION;
  DELETE FROM flmstudents WHERE studentid = 3;
COMMIT;   -- Now this specific deletion is saved
```

Output

```
Rollback successful.
All 3 students still exist — the accidental DELETE was undone.
```

IMPORTANT

ROLLBACK only works if you used BEGIN TRANSACTION before your statements. Without BEGIN TRANSACTION, SQL Server auto-commits each statement immediately, and ROLLBACK cannot undo it.

3.3 TCL Quick Comparison

Command	Purpose	Can Undo?
BEGIN TRANSACTION	Start a new transaction	N/A
COMMIT	Save all changes permanently	No

ROLLBACK	Undo all changes in transaction	Yes (before COMMIT)

4. Real-Time Interview Questions & Answers

These questions are commonly asked in SQL interviews for fresher and junior developer roles. Study them carefully!

4.1 WHERE Clause Questions

Q: What is the WHERE clause in SQL?

A: The WHERE clause filters rows in a SELECT, UPDATE, or DELETE statement based on a condition. Only rows that match the condition are returned or affected. Without WHERE, all rows in the table are processed.

Q: What is the difference between AND and OR?

A: AND returns rows only when ALL conditions are true (stricter filter). OR returns rows when AT LEAST ONE condition is true (broader filter). Example: WHERE city = 'Hyd' AND age > 20 — both must match. WHERE city = 'Hyd' OR city = 'Mumbai' — either city matches.

Q: What is the difference between IN and BETWEEN?

A: IN checks if a value matches any item in a specific LIST (e.g., IN ('Hyd', 'Mumbai', 'Delhi')). BETWEEN checks if a value falls within a RANGE (e.g., BETWEEN 60 AND 90). IN is for discrete values; BETWEEN is for continuous ranges.

Q: What is the LIKE operator? What are wildcards?

A: LIKE is used for pattern matching on text columns. It uses two wildcards: % (percent) matches zero or more characters, and _ (underscore) matches exactly one character. Example: LIKE 'R%' matches 'Rahul', 'Ravi', 'Ram'. LIKE '_a%' matches any name where the second character is 'a'.

Q: What is the difference between = and LIKE?

A: = is used for exact match comparison (e.g., WHERE name = 'Rahul'). LIKE is used for pattern matching with wildcards (e.g., WHERE name LIKE 'Ra%'). LIKE is slower than = for large tables since it scans for patterns.

Q: Can you combine AND, OR, and NOT in one query?

A: Yes! Use parentheses to control the order of evaluation. Example: SELECT * FROM students WHERE (city = 'Hyd' OR city = 'Mumbai') AND NOT marks < 50; — this gets students from Hyd or Mumbai who scored 50 or above. Without parentheses, AND has higher precedence than OR.

4.2 DCL Questions

Q: What are DCL commands? Name them.

A: DCL stands for Data Control Language. DCL commands control user access and permissions in a database. The main commands are GRANT (give permissions) and REVOKE (remove permissions). Some also include DENY (explicitly block access).

Q: What is the difference between GRANT and REVOKE?

A: GRANT gives a user permission to perform an action (e.g., SELECT, INSERT). REVOKE removes a previously granted permission. After REVOKE, the user loses that access. Example: GRANT SELECT ON students TO john — gives read access. REVOKE SELECT ON students FROM john — removes that access.

Q: What is a role in SQL? Why use roles?

A: A role is a named set of permissions that can be assigned to multiple users at once. Instead of granting permissions to each user individually, you assign a role. This simplifies management: if you add a permission to a role, all users with that role automatically get it. Common SQL Server roles: db_datareader, db_datawriter, db_owner.

Q: What is the principle of least privilege?

A: The principle of least privilege means giving users only the minimum permissions they need to do their job. For example, a reporting user should only have SELECT permission — not INSERT or DELETE. This reduces the risk of accidental or malicious data changes.

4.3 TCL Questions

Q: What is a transaction in SQL?

A: A transaction is a group of SQL statements that execute together as a single unit. Either all statements succeed (and changes are committed) or all fail (and changes are rolled back). Transactions ensure data consistency — for example, a bank transfer must deduct from one account AND add to another — both or neither.

Q: What is the difference between COMMIT and ROLLBACK?

A: COMMIT permanently saves all changes made in the current transaction to the database. Once committed, changes cannot be undone. ROLLBACK undoes all changes made in the current transaction, restoring the database to its state before the transaction started. ROLLBACK only works before a COMMIT.

Q: What happens if a transaction is not committed or rolled back?

A: If a transaction is neither committed nor rolled back (e.g., the application crashes), SQL Server automatically rolls back the incomplete transaction to maintain data integrity. Changes are lost and the database reverts to its pre-transaction state.

Q: Can you rollback a COMMIT?

A: No — once you execute COMMIT, the changes are permanently saved and cannot be undone with ROLLBACK. You would need to manually write a new SQL statement to reverse the changes (e.g., DELETE the inserted row, or UPDATE it back to the original value).

Q: Scenario: You ran DELETE without WHERE and committed. How do you recover the data?

A: Once committed, ROLLBACK will not work. Recovery options are: (1) Restore from a database backup. (2) Use transaction log backups (if available) to restore to a point in time. (3) If using SQL Server Enterprise, use features like point-in-time restore. This is why ALWAYS taking backups and testing queries in a dev environment is critical.

5. Quick Revision Cheat Sheet

```
-- ========== WHERE CLAUSE ==========

SELECT * FROM flmstudents WHERE address = 'Hyderabad';              --
Basic filter
SELECT * FROM flmstudents WHERE address = 'Hyd' AND marks > 60;     -- AND
SELECT * FROM flmstudents WHERE address = 'Hyd' OR address = 'Mum'; -- OR
SELECT * FROM flmstudents WHERE NOT address = 'Delhi';               --
NOT
SELECT * FROM flmstudents WHERE address IN ('Hyd', 'Mum', 'Del');   -- IN
SELECT * FROM students WHERE marks BETWEEN 60 AND 90;               --
BETWEEN
SELECT * FROM flmstudents WHERE studentname LIKE 'R%';              --
LIKE starts
SELECT * FROM flmstudents WHERE emailid LIKE '%gmail.com';          --
LIKE ends

-- ========== DCL ==========

GRANT SELECT ON flmstudents TO john;                -- Give read access
GRANT SELECT, INSERT, UPDATE ON flmstudents TO mgr; -- Multiple
permissions
REVOKE SELECT ON flmstudents FROM john;             -- Remove read access
CREATE ROLE student_viewer;                         -- Create a role
GRANT SELECT ON flmstudents TO student_viewer;      -- Assign perms to
role
ALTER ROLE student_viewer ADD MEMBER john;          -- Add user to role
ALTER ROLE student_viewer DROP MEMBER john;         -- Remove user from
role

-- ========== TCL ==========

BEGIN TRANSACTION;                          -- Start transaction
  INSERT INTO flmstudents VALUES (...);     -- Statement 1
  UPDATE flmstudents SET ... WHERE ...;     -- Statement 2
COMMIT;                                     -- Save permanently

BEGIN TRANSACTION;
  DELETE FROM flmstudents;                  -- Dangerous!
ROLLBACK;                                   -- Undo everything
```